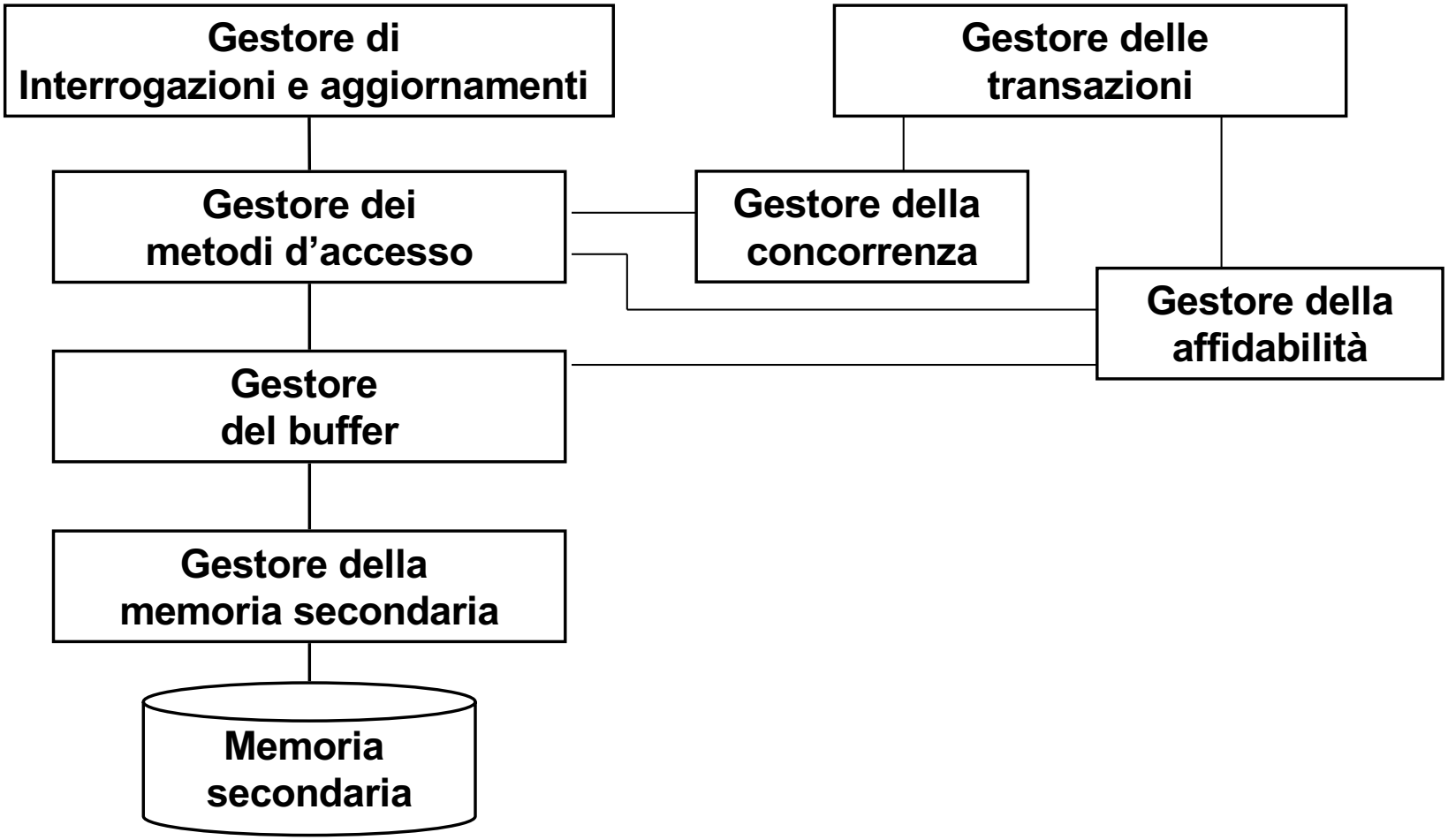
Gestione dell'affidabilità (atomicità e durabilità)

Gestore degli accessi e delle interrogazioni

Gestore delle transazioni



Persistenza delle memorie

- Memoria centrale: non è persistente
- Memoria di massa: è persistente ma può danneggiarsi
- **Memoria stabile**: memoria che non può danneggiarsi
  - astrazione "ideale," non esiste, ma
  - perseguita attraverso la ridondanza:
    - dischi replicati
    - nastri
    - …

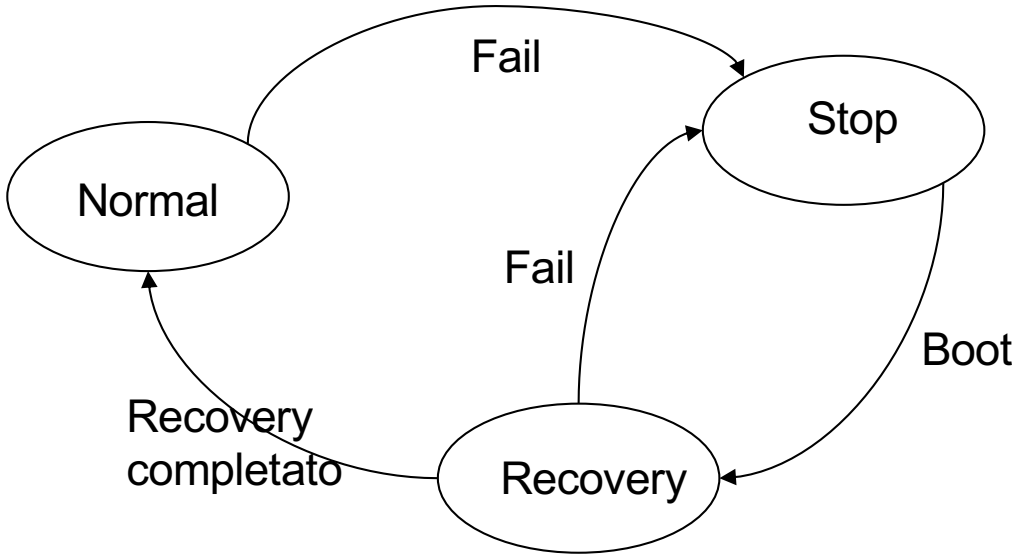
Guasti

- **Guasti "soft":** errori di programma, crash di sistema, caduta di tensione
  - si perde la memoria centrale
  - non si perde la memoria secondaria

  warm restart (recovery), ripresa a caldo
- **Guasti "hard":** sui dispositivi di memoria secondaria
  - si perde anche la memoria secondaria
  - non si perde la memoria stabile

  cold restart, ripresa a freddo
- **Catastrofe:**
  - perdita di memoria stabile
  - supponiamo che non si possa verificare

Modello "fail-stop"



Gestore dell'affidabilità

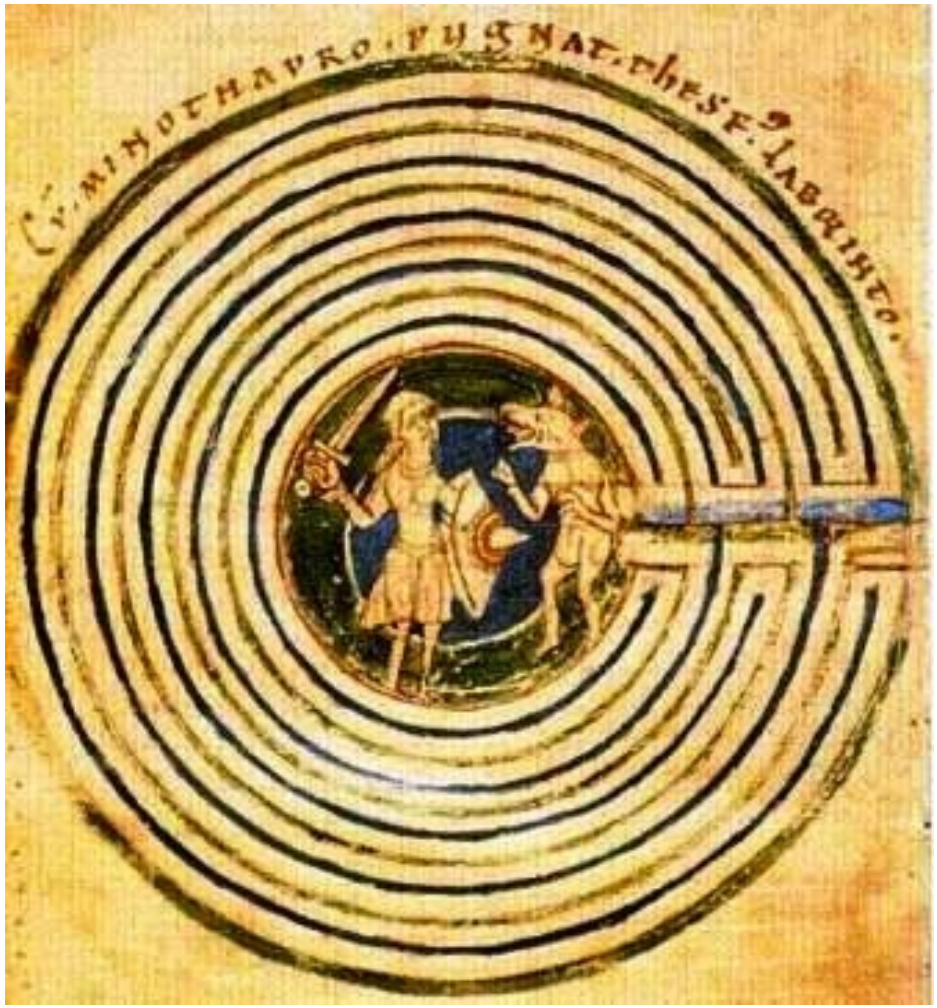
- Assicura atomicità e durabilità
- Gestisce l'esecuzione dei comandi transazionali
  - **`start transaction`** (B, begin)
  - **`commit work`** (C)
  - **`rollback work`** (A, abort)

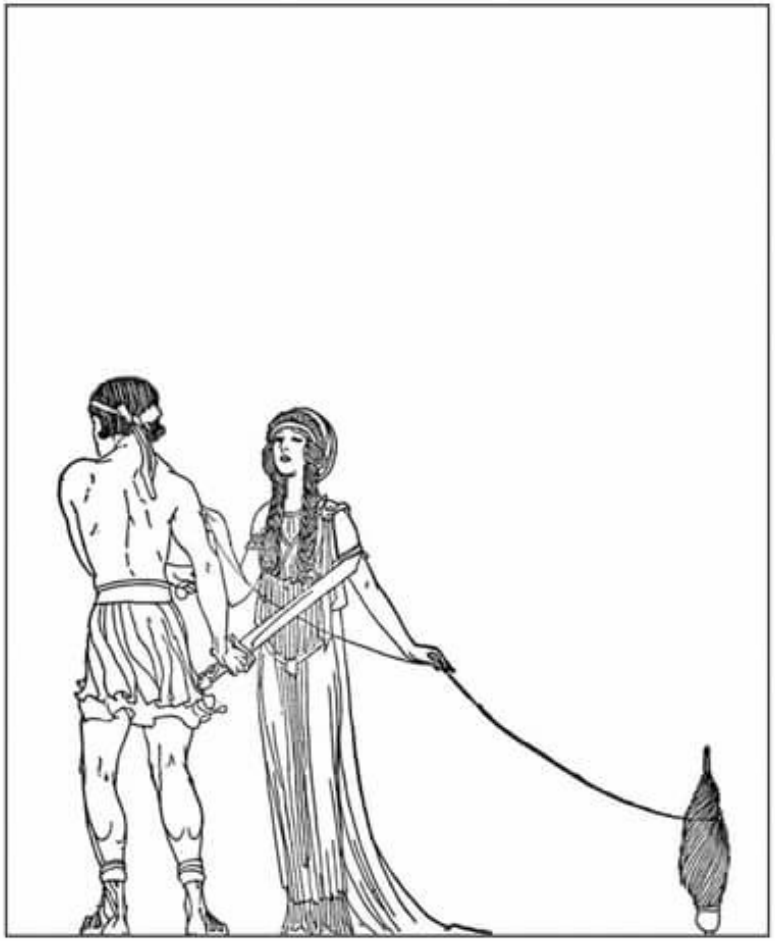
  e le operazioni di ripristino (recovery) dopo i guasti:
  - *warm restart* e *cold restart*

Gestore dell'affidabilità: usa informazioni ridondanti

- **log**
  - un archivio permanente (in memoria stabile) che registra le operazioni svolte
- **dump**
  - copia di riserva della base di dati (pure in memoria stabile)

Il filo di Arianna



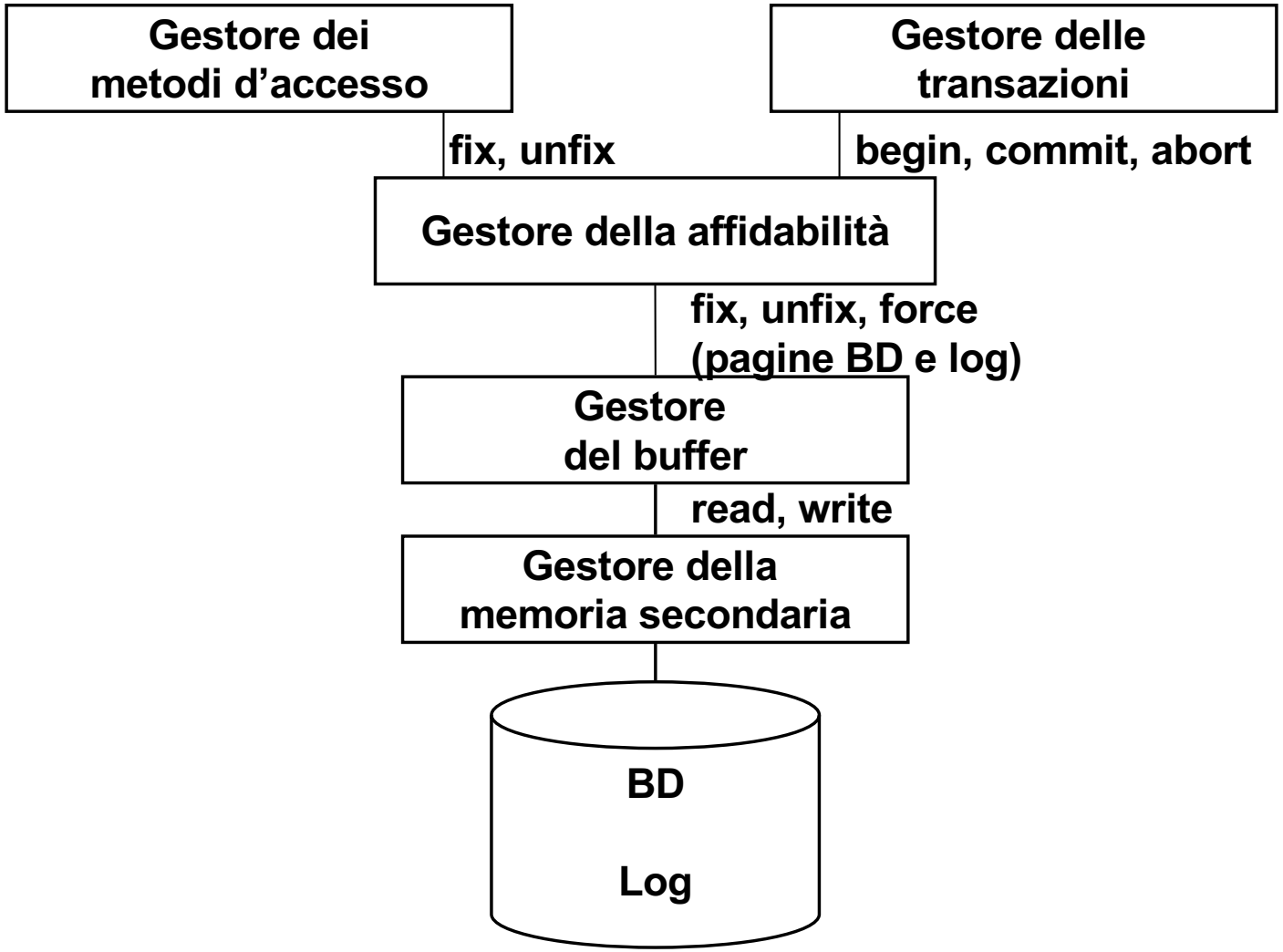


Hansel e Gretel

Gestore dell'affidabilità: usa informazioni ridondanti

- **log** (o "giornale" o "diario di bordo")
  - un archivio permanente che registra le operazioni svolte
    - è un file sequenziale, riporta tutte le operazioni in ordine
    - scritto in memoria stabile, non sempre subito, ma sempre in ordine
  - a che cosa serve? Ad esempio a tornare indietro, come:
    - il filo di Arianna
    - i sassolini e le briciole di Hansel e Gretel
      - che differenza c'è fra sassolini e briciole?
- **dump**
  - copia di riserva della base di dati
    - un'altra forma di ridondanza: un "punto di partenza"

Architettura del controllore dell'affidabilità



Modello di riferimento

- Una transazione è costituita da una sequenza di operazioni di input-output su "oggetti" astratti
  - oggetto: un blocco, un record, …
  - operazioni
    - I(O): inserimento di un nuovo oggetto O
    - D(O): eliminazione (delete) di O
    - U(O): aggiornamento (update) di O
  - ignoriamo la semantica di dettaglio delle applicazioni, ci interessano solo i valori
    - "prima" dell'operazione (BS, *before state*)
      - per eliminazione e aggiornamento
    - "dopo" l'operazione (AS, *after state*)
      - per inserimento e aggiornamento

Che cosa scriviamo nel log

- Record nel log
  - operazioni delle transazioni (T identificatore della transazione, O oggetto, BS before state, AS after state)
    - begin, B(T)
    - insert, I(T,O,AS)
    - delete, D(T,O,BS)
    - update, U(T,O,BS,AS)
    - commit, C(T), abort, A(T)
  - record di sistema (vediamo dopo)
    - dump
    - checkpoint

Esito di una transazione

- Quando si conclude positivamente una transazione?
  - quando viene scritto il record di **commit** nel log
  - e questo deve essere **irrevocabile**
- Quindi:
  - il commit non si può perdere:
    - va scritto subito, **in memoria stabile**, con una **force**

- Nota
  - I record di abort possono essere scritti in modo asincrono

Esito di una transazione (2)

- Conseguenze dell'irrevocabilità
  - un guasto prima del commit può portare la transazione a non concludersi
    - serve ricostruire lo stato originario della base di dati
  - un guasto successivo non deve avere conseguenze:
    - lo stato finale della base di dati deve essere ricostruito (se non si è certi che sia stato ottenuto)
- Quindi
  - servono meccanismi per annullare e ricostruire

Undo e redo

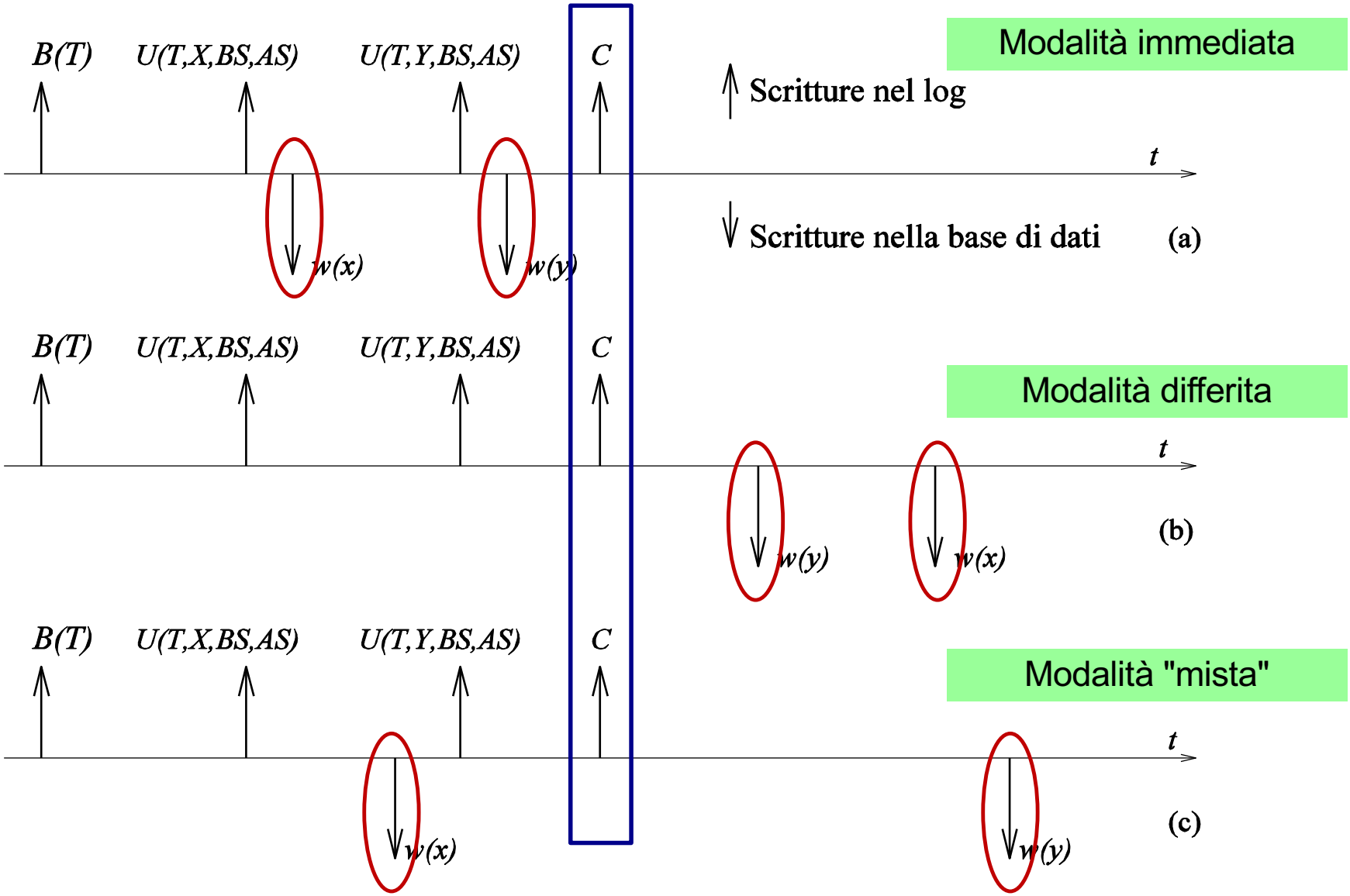
- Undo di una azione su un oggetto O:
  - update, delete: copia il valore del before state (BS) nell'oggetto O
  - insert: eliminare O (se c'è)
- Redo di una azione su un oggetto O:
  - insert, update: copia il valore dell' after state (AS) nell'oggetto O
  - delete: elimina O (se c'è)

- Idempotenza di undo e redo (vedremo fra poco l'importanza)
  - undo(undo(A)) = undo(A)
  - redo(redo(A)) = redo(A)

Regole fondamentali per il log

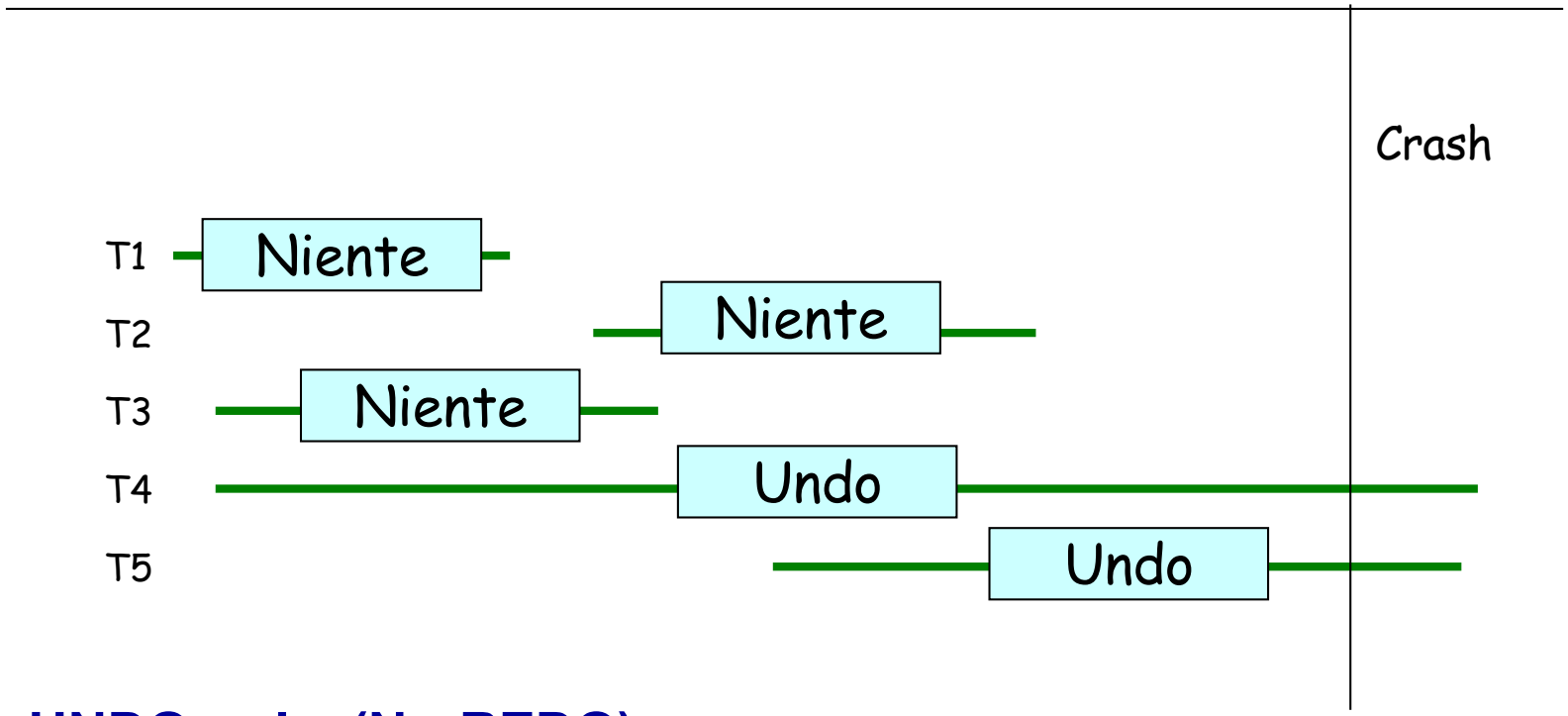
- Concetto base
  - prima scriviamo nel log e poi facciamo il resto (commit e scrittura nella base di dati)
    - "prendete nota della password prima di procedere"
- Il log deve contenere le informazioni necessarie a ripristinare lo stato corretto in caso di guasto
  - **dobbiamo poter "disfare" le azioni**
    - **Write-Ahead-Log**: si scrive il BS sul log (in memoria stabile) prima di scrivere nella base di dati
  - **dobbiamo poter "rifare" le azioni**
    - **Commit-Precedenza**: si scrive l'AS sul log (in memoria stabile) prima del commit
      - "scrivetevi la password prima di modificarla"
- Quando scriviamo nella base di dati?
  - varie alternative

Scrittura nel log e nella base di dati



Modalità immediata

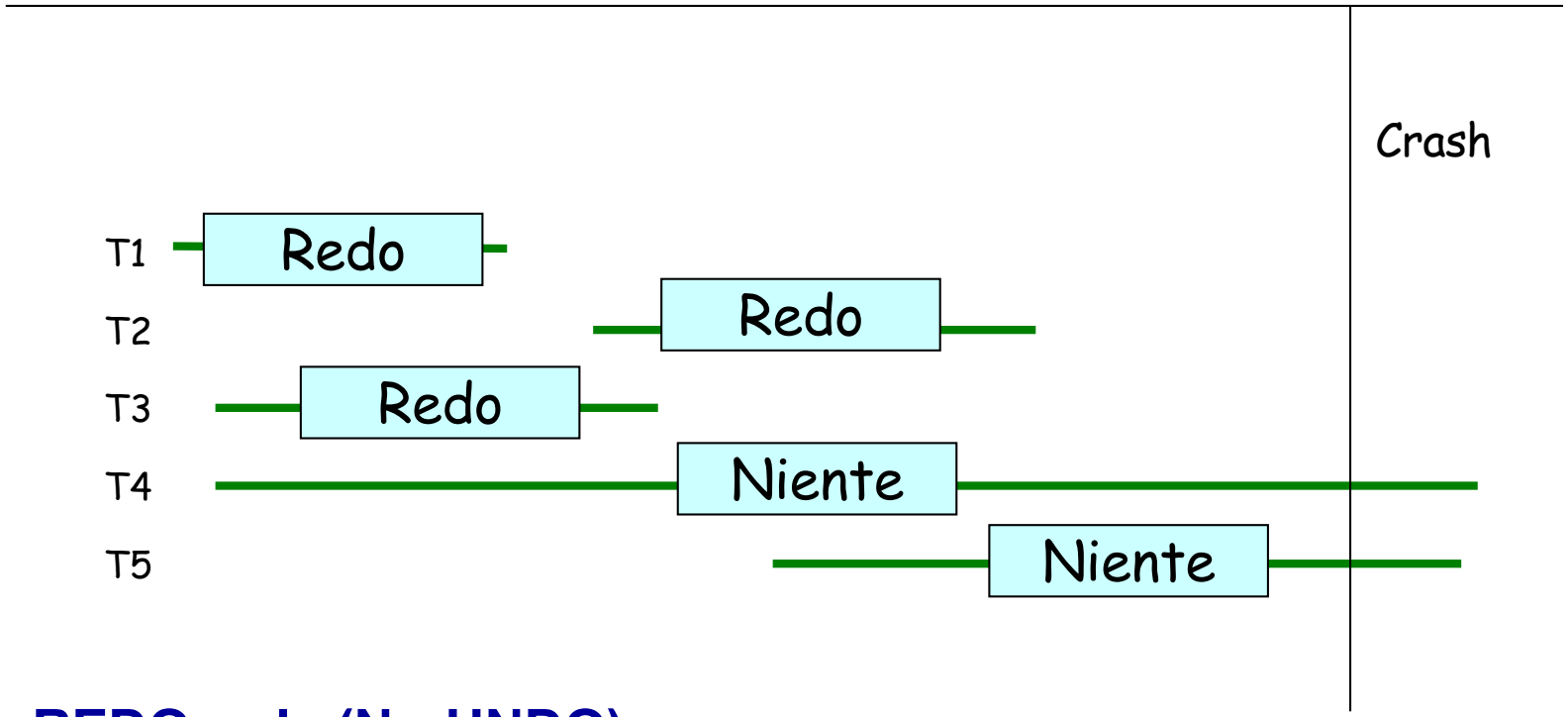
- Il DB contiene valori modificati per transazioni committed (tutte) e uncommitted



- **UNDO-only (No-REDO)**

Modalità differita

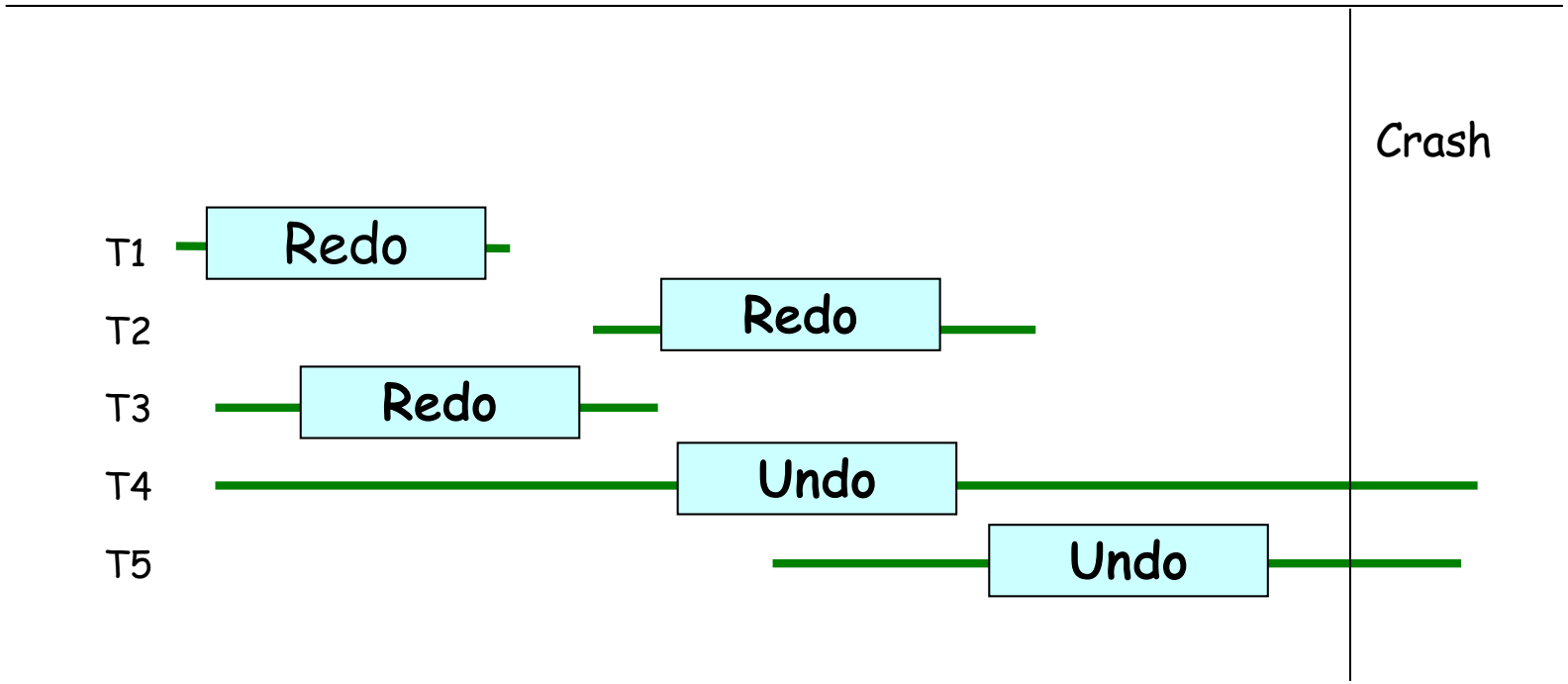
- Il DB non contiene valori modificati da transazioni uncommitted, ma non è sicuro contenga i nuovi valori per quelle committed



- **REDO-only (No-UNDO)**

Esiste una terza modalità: modalità mista

- La scrittura può avvenire in modalità immediata e/o differita



- **REDO - UNDO**

Quale conviene?

- La terza, perché
  - anche se richiede più operazioni in caso di guasto
  - permette al gestore del buffer di decidere quando scrivere, privilegiando l'operatività ordinaria (salvo il caso di guasto)